

```

// Global Variables
CONSTANT NullPointer <- 0
TYPE Tree
    DECLARE LeftPointer : INTEGER
    DECLARE RightPointer : INTEGER
    DECLARE Data : CHAR
ENDTYPE

DECLARE RootPointer, FreePointer : INTEGER
DECLARE MyTree : ARRAY[1:7] OF Tree

// Initialise a tree
PROCEDURE CreateTree()
    DECLARE Index : INTEGER
    RootPointer <- NullPointer
    FreePointer <- 1
    FOR Index = 1 TO 6
        MyTree[Index].RightPointer <- Index + 1
    NEXT Index
    MyTree[7].RightPointer <- NullPointer
ENDPROCEDURE

// Add an item in tree
PROCEDURE AddItem(NewItem : CHAR)
    DECLARE CurrentPointer : INTEGER
    DECLARE Found, turnRight : BOOLEAN
    IF FreePointer = NullPointer THEN
        OUTPUT "Tree is full"
    ELSE
        Temp <- FreePointer
        FreePointer <- MyTree[Temp].RightPointer
        // adding data to the current free pointer
        MyTree[Temp].Data <- NewItem
        MyTree[Temp].LeftPointer <- NullPointer
        MyTree[Temp].RightPointer <- NullPointer
        IF RootPointer = NullPointer THEN // if its the first data make fp and rp same
            RootPointer <- Temp
        ELSE
            // Traversing thorough the list
            CurrentPointer <- RootPointer
            WHILE CurrentPointer <> NullPointer
                PreviousPointer <- CurrentPointer
                IF NewItem > MyTree[CurrentPointer].Data THEN
                    turnRight <- True
                    CurrentPointer <- MyTree[CurrentPointer].RightPointer
                ELSE
                    turnRight <- False
                    CurrentPointer <- MyTree[CurrentPointer].LeftPointer
                ENDWHILE
            IF turnRight = True THEN
                MyTree[PreviousPointer].RightPointer <- Temp
            ELSE
                MyTree[PreviousPointer].LeftPointer <- Temp
            ENDIF
        END
    ENDIF

// Search Item
FUNCTION SearchItem(SearchItem : CHAR) RETURN INTEGER
    DECLARE CurrentPointer : INTEGER
    DECLARE Found : BOOLEAN
    Found <- False
    CurrentPointer <- RootPointer
    WHILE NOT Found AND CurrentPointer <> NullPointer
        IF SearchItem = MyTree[CurrentPointer].Data THEN
            Found <- True
        ENDIF
    ENDWHILE
ENDFUNCTION

```

```
        ELSEIF SearchItem > MyTree[CurrentPointer] THEN
            CurrentPointer <- MyTree[CurrentPointer].RightPointer
        ELSE
            CurrentPointer <- MyTree[CurrentPointer].LeftPointer
        ENDIF
    ENDWHILE
    RETURN CurrentPointer
END
```